

Exponential Backoff and Retry

Definition

Exponential Backoff and Retry is a fault-tolerance technique where a client retries failed requests, but waits longer after each failure.

The retry delay increases exponentially instead of retrying immediately.

Simple Meaning

If request fails, retry later... and wait longer each time.

Why Needed

Without backoff:

- Immediate repeated retries
- Service overload gets worse
- Retry storm
- Higher latency
- Cascading failures

With exponential backoff:

- Gives downstream service time to recover
- Reduces pressure on failing systems
- Improves success chance later
- Better resource usage

Basic Retry Example

Normal retries:

None

Try -> Fail

Try -> Fail

Try -> Fail

Immediate retries create load.

Exponential Backoff Example

None

1st retry after 1 sec

2nd retry after 2 sec

3rd retry after 4 sec

4th retry after 8 sec

5th retry after 16 sec

Formula:

None

$\text{delay} = \text{base} * 2^{\text{attempt}}$

Example:

- base = 1 second

What is Jitter?

Jitter = random small variation added to retry delay.

Example:

Instead of all clients retrying exactly at 4 sec:

- Client A → 3.7 sec
- Client B → 4.2 sec
- Client C → 4.6 sec

Why Jitter Important

Without jitter:

Thousands of clients retry together.

This causes:

- Retry storm
- Traffic spikes
- Another outage

With jitter:

Retries spread over time.

Real Example

Checkout service calls Payment API.

Payment API returns 503.

Retry logic:

1. Retry after 1 sec
2. Retry after 2 sec
3. Retry after 4 sec
4. Retry after 8 sec
5. Fail permanently

Success Reset Behavior

If request succeeds:

- Reset retry counter
- Resume normal traffic

Where Used

1. API Calls

- Payment gateways
- Email providers
- SMS services

2. Microservices

Service A calling Service B

3. Database Connections

Temporary DB outage

4. Message Queues

Consumer processing failures

Retryable Errors

Good for temporary errors:

- 429 Too Many Requests
- 500 Internal Error
- 502 Bad Gateway
- 503 Service Unavailable
- Timeout
- Network issue

Non-Retryable Errors

Do NOT retry:

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- Validation errors
- Wrong payload

Key Parameters

Parameter	Meaning
-----------	---------

Base delay	Initial wait time
Max delay	Maximum wait
Max retries	Retry attempts limit
Jitter	Random variance

Example Strategy

Attempt	Delay
1	1 sec
2	2 sec
3	4 sec
4	8 sec
5	Stop

Benefits

- 1. Reduces Load During Failure
- 2. Handles Temporary Errors
- 3. Improves Reliability
- 4. Prevents Retry Storm (with jitter)

Tradeoffs

Advantage	Tradeoff
Better recovery chance	Slower final response
Less overload	Added complexity
Works well for transient failures	Not for permanent failures

Retry vs Circuit Breaker

Retry	Circuit Breaker
-------	-----------------

Retry failed request	Stop requests after repeated failures
Best for temporary issues	Best for persistent outages

Best together.

Combined Pattern

1. Retry 3 times with exponential backoff
2. If many failures continue
3. Open circuit breaker

Interview Answer

I'd use exponential backoff with jitter for temporary failures such as 503 or timeouts. Retries occur after increasing delays like 1s, 2s, 4s, 8s to avoid overwhelming the downstream service.

Idempotency Note

Safe retries usually require idempotent operations.

Example:

POST payment must use idempotency key.

Real-World Examples

Amazon Web Services SDKs often support retry strategies.
Google APIs commonly recommend exponential backoff.

One-Line Memory Trick

Fail once = retry soon, fail repeatedly = retry slower.